

Mock Interview 3

Advanced Round — Architecture, System Design & Leadership

Candidate	Viraj Pawar
Experience	3.5 Years Java Full Stack (Capgemini)
Duration	55–65 Minutes
Difficulty	■■■■ Hard
Stack	Java · Spring Boot · React · MySQL · Docker · K8s

Sections Covered in This Mock Interview

Section 1	HR + Strategic Behavioral Round (10 Questions)
Section 2	Core Java Deep Dive — Internals & Edge Cases (12 Questions)
Section 3	Spring Boot Architecture + Production Scenarios (14 Questions)
Section 4	JPA + Database — Query Optimization (10 Questions)
Section 5	System Design — Full Job Portal Redesign Discussion
Section 6	DevOps — Cloud, Kubernetes, SRE Concepts (10 Questions)
Section 7	Coding Round (4 Problems — Medium to Hard)
Section 8	Rapid Fire — Tricky Cross Questions (15 Questions)
Section 9	Evaluation Guide — Senior-Level Expectations

■ SECTION 1: HR + STRATEGIC BEHAVIORAL ROUND

Q1.

How do you approach technical decisions that have long-term implications?

◆ **Model Answer**

I use a lightweight Architecture Decision Record (ADR) process. For any decision with significant long-term impact (choosing a message broker, database technology, caching strategy), I document: Context, Options considered, Decision made, Consequences. I present the ADR to senior engineers for review before committing. This creates institutional memory and prevents revisiting the same decisions repeatedly. In my Job Portal, I wrote ADRs for: database-per-service pattern, REST vs messaging for inter-service communication, and JWT vs session-based auth.

Q2.

How do you handle technical debt in a fast-paced environment?

◆ **Model Answer**

I advocate for making technical debt visible. I create JIRA tickets for tech debt with a business impact description (e.g., 'No DB indexes on applications.applied_date — query slows down by 2s per 10K records'). I negotiate 20% of each sprint for tech debt — which most product managers accept when they understand the performance cost of ignoring it. I prioritize debt that causes production incidents first, then reliability improvements, then clean-up.

Q3.

How have you influenced technical direction on your team?

◆ **Model Answer**

When our team was using String concatenation in a loop for building complex SQL query fragments in a reporting module, I noticed it during code review. Instead of just fixing it, I organized a 20-minute knowledge session showing the performance difference between String + vs StringBuilder and introduced a team-wide coding standard. I also configured a SonarQube rule to flag this pattern. The team adopted it and it became part of our Definition of Done.

Q4.

Describe a situation where you had to balance technical quality with business needs.

◆ **Model Answer**

The product team wanted the 'Apply for Job' feature in 3 days. The proper implementation needed resume parsing, email notification, and employer dashboard update — complex. I proposed a phased approach: Day 3 — basic application submission with status tracking (90% of the user value). Week 2 — add resume parsing. Week 3 — employer notification. The business got their feature launch, and I delivered a stable, extensible foundation without cutting corners on core data integrity.

Q5.

What is your approach to code reviews?

◆ **Model Answer**

I follow a constructive, empathy-first approach. I focus on: (1) correctness first — will this break in edge cases? (2) readability — can a new team member understand this in 6 months? (3) performance — are

there obvious inefficiencies like N+1 or missing index? I always explain WHY I'm suggesting a change, not just WHAT to change. I also acknowledge what's done well. I aim to unblock PRs within 24 hours — a slow review process is a team velocity killer.

■ SECTION 2: CORE JAVA — INTERNALS & EDGE CASES

Q1.

Explain how the JVM works — ClassLoader, JIT, GC in brief.

◆ **Model Answer**

ClassLoader: Loads .class bytecode into the JVM using delegation model — Bootstrap → Platform → Application ClassLoader. Custom ClassLoaders allow dynamic loading (plugins, hot reload). JIT (Just-In-Time) Compiler: Interprets bytecode initially. Hotspot detection — methods called frequently (>10,000 times by default) are JIT-compiled to native code. C1 (fast compile) and C2 (aggressive optimize). Java 17+ has GraalVM for ahead-of-time compilation to native. GC: Young Gen (Eden → Survivor), Old Gen. GC pauses are STW (Stop-The-World). G1GC (default Java 9+) minimizes pause time. ZGC (Java 15+) aims for sub-millisecond pauses.

Q2.

What happens during a ConcurrentModificationException? How do you avoid it?

```
// Problem: modifying list while iterating List<String> list = new
ArrayList<>(Arrays.asList("a","b","c")); for (String s: list) { if
(s.equals("b")) list.remove(s); // ConcurrentModificationException! } // Fix 1:
Iterator.remove() Iterator<String> it = list.iterator(); while (it.hasNext()) { if
(it.next().equals("b")) it.remove(); // safe } // Fix 2: Java 8 removeIf
list.removeIf(s -> s.equals("b")); // Fix 3: Stream filter (creates new list) list
= list.stream().filter(s -> !s.equals("b")).collect(Collectors.toList());
```

Q3.

What is String interning and when would you use it explicitly?

◆ **Model Answer**

The JVM maintains a String pool in the heap (moved from PermGen in Java 7). String literals are automatically interned — 'abc' == 'abc' is true because they reference the same pool entry. String.intern() explicitly adds a string to the pool and returns the canonical instance. Use case: If you have 1 million duplicate String values (e.g., country codes) in memory, interning reduces memory from N objects to 1 object per unique value. Caution: The pool is bounded by heap space — excessive interning can cause memory issues.

Q4.

What is the difference between Heap and Off-Heap memory in Java?

◆ **Model Answer**

Heap memory: Managed by GC. All regular Java objects live here. GC pauses affect latency. Off-Heap: Managed manually via java.nio.ByteBuffer.allocateDirect() or Unsafe. Not subject to GC — useful for low-latency applications (trading systems, high-throughput caches). Also used by NIO buffers, Ehcache off-heap mode, and databases like Cassandra. Risk: Memory leaks are possible since GC doesn't clean up — must call cleaner explicitly.

Q5.

Explain the difference between Parallel Streams and Virtual Threads (Java 21).

◆ **Model Answer**

Parallel Streams: Use the `ForkJoinPool.commonPool()`. Good for CPU-bound, data-parallel operations on collections. Not suitable for blocking I/O — threads get blocked, starving the pool. Virtual Threads (Java 21): Ultra-lightweight threads managed by JVM. Millions can exist concurrently. When a virtual thread blocks on I/O, it unmounts from the OS thread — no wasted resources. Perfect for I/O-bound workloads like REST APIs, JDBC calls. Spring Boot 3.2+ supports virtual threads: `spring.threads.virtual.enabled=true`.

Q6.

What is weak vs soft vs phantom reference in Java?

◆ Model Answer

Strong reference: Default. Object not GC'd while referenced. SoftReference: GC'd only when JVM needs memory (good for in-memory caches). WeakReference: GC'd at next GC cycle, regardless of memory (used in WeakHashMap for canonical mappings). PhantomReference: Enqueued to a ReferenceQueue AFTER GC — used for post-finalization cleanup, more reliable than `finalize()`.

Q7.

What is the double-checked locking singleton pattern and what's the issue?

```
// BROKEN without volatile: public class Singleton { private static Singleton
instance; // NOT volatile public static Singleton getInstance() { if (instance ==
null) { synchronized (Singleton.class) { if (instance == null) instance = new
Singleton(); // reordering risk! } } return instance; } } // FIXED with volatile
(prevents instruction reordering): private static volatile Singleton instance; //
BEST: Initialization-on-demand holder (no synchronization needed) public class
Singleton { private Singleton() {} private static class Holder { static final
Singleton INSTANCE = new Singleton(); } public static Singleton getInstance() {
return Holder.INSTANCE; } }
```

Q8.

What is ThreadLocal and when would you use it?

```
// ThreadLocal provides thread-specific variables public class RequestContext {
private static final ThreadLocal<String> CORRELATION_ID = new ThreadLocal<>();
public static void setCorrelationId(String id) { CORRELATION_ID.set(id); } public
static String getCorrelationId() { return CORRELATION_ID.get(); } // CRITICAL:
Always clean up to prevent memory leaks in thread pools! public static void
clear() { CORRELATION_ID.remove(); } }
```

◆ Model Answer

Use case: Passing correlation IDs, user context, or database connections across method calls within the same thread without passing them as parameters. Spring Security uses ThreadLocal for SecurityContextHolder. Warning: Always call `.remove()` when done — thread pool threads are reused and stale values can leak between requests.

■ SECTION 3: SPRING BOOT — PRODUCTION SCENARIOS

Q1.

Scenario: Your service is leaking memory and you get `OutOfMemoryError` after 24 hours of running. How do you debug it?

◆ **Model Answer**

Step 1: Take a heap dump when memory is high: `jmap -dump:format=b,file=heap.hprof`. Step 2: Analyze with Eclipse MAT — look for dominator tree (what's holding most memory). Step 3: Common causes: (a) Static collections growing without eviction (cache without TTL). (b) `ThreadLocal` not cleaned up — leaks in request context. (c) `ClassLoader` leak in dynamic class loading. (d) Event listeners not deregistered — holding references to objects. Step 4: Enable `-verbose:gc -Xloggc:gc.log` to see GC frequency and heap growth rate. Step 5: Use Spring Boot Actuator `/actuator/metrics/jvm.memory.used` to monitor over time.

Q2.

How would you implement distributed locking in a multi-instance Spring Boot deployment?

◆ **Model Answer**

Problem: If you have 3 instances of your service running and a scheduled task runs on all 3 simultaneously, you get duplicate processing. Solution: Distributed lock using Redis (Redisson) or using a DB-based lock. Redis approach with Redisson:

```
@Autowired RedissonClient redissonClient; public void processScheduledJobs() {
    RLock lock = redissonClient.getLock("scheduled-job-lock"); boolean acquired =
    false; try { acquired = lock.tryLock(0, 30, TimeUnit.SECONDS); // TTL: 30s if
    (acquired) { doProcessJobs(); // only one instance runs this } } catch
    (InterruptedException e) { Thread.currentThread().interrupt(); } finally { if
    (acquired && lock.isHeldByCurrentThread()) lock.unlock(); } }
```

Q3.

How do you implement an audit log for all entity changes using Spring?

```
// Option 1: Spring Data JPA Auditing
@EntityListeners(AuditingEntityListener.class) @MappedSuperclass public abstract
class Auditable { @CreatedBy String createdBy; @LastModifiedBy String modifiedBy;
@CreatedDate LocalDateTime createdAt; @LastModifiedDate LocalDateTime updatedAt; }
// Option 2: Hibernate Envers for full revision history @Entity @Audited // tracks
every change with revision numbers public class Job extends Auditable {} // Query
revision history: AuditReader reader = AuditReaderFactory.get(entityManager);
List<Job> revisions = reader.createQuery().forRevisionsOfEntity(Job.class, true,
true).add(AuditEntity.id().eq(jobId)).getResultList();
```

Q4.

How do you handle graceful shutdown in Spring Boot with in-flight requests?

```
# application.properties: server.shutdown=graceful # enables graceful shutdown
spring.lifecycle.timeout-per-shutdown-phase=30s # wait up to 30s # With
Docker/K8s, also add: # terminationGracePeriodSeconds: 40 in deployment spec #
(must be > Spring timeout to avoid SIGKILL) // Custom pre-destroy hook:
@PreDestroy public void onShutdown() { log.info("Shutting down gracefully..."); //
close external connections, flush buffers }
```

◆ Model Answer

K8s sends SIGTERM → Spring waits for in-flight requests to complete (up to timeout) → Spring closes → K8s sends SIGKILL if still running after terminationGracePeriodSeconds. Set terminationGracePeriodSeconds slightly higher than Spring's timeout.

Q5.

How do you implement request-scoped caching (same data per HTTP request)?

```
// Use @RequestScope bean – created fresh per HTTP request @Component
@RequestScope public class RequestCache { private final Map<String, Object> cache
= new HashMap<>(); public <T> T getOrCompute(String key, Supplier<T> loader) {
return (T) cache.computeIfAbsent(key, k -> loader.get()); } } // Usage: same user
data within one request is fetched only once @Service public class
ApplicationService { @Autowired RequestCache reqCache; public void process(Long
userId) { UserDTO user = reqCache.getOrCompute("user:" + userId, () ->
userService.findById(userId)); } }
```

Q6.

Explain Spring's event publishing mechanism. How is it different from async messaging?

```
// Synchronous Spring events (same JVM, same thread by default) @Service public
class ApplicationService { @Autowired ApplicationEventPublisher publisher; public
void applyForJob(Long jobId, Long userId) { // ... save application
publisher.publishEvent(new JobAppliedEvent(this, jobId, userId)); // Listener
called synchronously here before return } } @Component public class
NotificationListener { @EventListener @Async // makes this listener async public
void onJobApplied(JobAppliedEvent event) {
emailService.sendConfirmation(event.getUserId()); } }
```

◆ Model Answer

Spring events: In-process, no infrastructure needed. Good for simple decoupling within one service. If the JVM dies, the event is lost. Messaging (Kafka/RabbitMQ): Cross-service, durable, replayable, supports consumer groups for scaling. Use Spring events for in-process coordination and messaging for cross-service communication.

■ SECTION 4: JPA + DATABASE — QUERY OPTIMIZATION

Q1.

How do you approach database query optimization end-to-end?

◆ **Model Answer**

Step 1: Identify slow queries — enable slow query log (long_query_time=2) in MySQL. Step 2: Run EXPLAIN ANALYZE on the slow query. Step 3: Add missing indexes (check where the full table scan is). Step 4: Check if the query can be rewritten — avoid SELECT *, avoid functions on indexed columns. Step 5: Consider denormalization for read-heavy queries. Step 6: Add Redis/Caffeine caching for repeated identical queries. Step 7: For analytics queries, consider a separate read replica or data warehouse.

Q2.

Write a complex query: Find the average time-to-hire (application submission to offer) by company.

```
SELECT c.name AS company, COUNT(a.id) AS total_hires,
ROUND(AVG(DATEDIFF(a.offer_date, a.applied_date)), 1) AS avg_days_to_hire,
MIN(DATEDIFF(a.offer_date, a.applied_date)) AS fastest_hire_days,
MAX(DATEDIFF(a.offer_date, a.applied_date)) AS slowest_hire_days FROM applications
a INNER JOIN jobs j ON a.job_id = j.id INNER JOIN companies c ON j.company_id =
c.id WHERE a.status = 'OFFERED' AND a.offer_date IS NOT NULL AND a.applied_date >=
DATE_SUB(NOW(), INTERVAL 12 MONTH) GROUP BY c.id, c.name HAVING COUNT(a.id) >= 5
-- only companies with 5+ hires for statistical significance ORDER BY
avg_days_to_hire ASC;
```

Q3.

What is the difference between clustered and non-clustered index?

◆ **Model Answer**

Clustered index: Determines the physical storage order of rows. Only one per table. In InnoDB (MySQL default), the PRIMARY KEY is always the clustered index. Data pages are physically ordered by PK. Non-clustered index: A separate structure pointing to data rows. Multiple allowed per table. For non-clustered index lookups, InnoDB first finds the PK in the index, then does a second lookup in the clustered index (bookmark lookup). For this reason, keeping PK values small (Long vs UUID) improves secondary index performance.

Q4.

What is a covering index and how did you use it?

◆ **Model Answer**

A covering index contains all columns needed by a query — the DB can answer the query from the index alone without reading the actual table rows. Example: If your query is SELECT title, salary FROM jobs WHERE status = 'ACTIVE' AND company_id = 5, a composite index on (status, company_id, title, salary) would cover this query completely. EXPLAIN output shows 'Using index' instead of 'Using index; Using table' for covered queries. In my project, I created covering indexes for the most frequent reporting queries.

Q5.

How do you handle schema migrations safely in production?

◆ Model Answer

Use Flyway for versioned migrations. Best practices: (1) Never modify an existing migration file — always add a new version. (2) For large table migrations, use pt-online-schema-change (Percona Tool) or gh-ost to avoid locking. (3) Additive changes first: add a nullable column in migration V1, deploy app code that writes to it, then in V2 add the NOT NULL constraint. (4) Always have a rollback migration file tested before applying. (5) Apply migrations in a separate pipeline step BEFORE deploying the new application version.

■ SECTION 5: SYSTEM DESIGN — JOB PORTAL REDESIGN

INTERVIEWER SCENARIO: 'Viraj, imagine you are now the lead engineer. You need to redesign your Job Portal to handle 1 million daily active users and 10 million job listings. Walk me through your architecture decisions.'

Q1.

How would you redesign the data storage layer for 10M job listings?

◆ **Model Answer**

Primary DB: Relational DB for transactional data (applications, users, companies) with read replicas. MySQL can handle this with proper indexing and partitioning. Search: Move job search from MySQL LIKE to Elasticsearch. ES handles full-text search, geo-queries ('jobs near me'), and complex filtering (salary range, skills) at scale. Partitioning: Partition the jobs table by company_id or region if it grows beyond 500M rows. Archiving: Move jobs posted > 2 years ago to cold storage (AWS S3 + Athena for analytics). Caching: Redis for hot job listings (top 1000 searched jobs). CDN: Pre-rendered popular search results cached at the edge.

Q2.

How would you design the notification system at scale?

◆ **Model Answer**

Event-driven with Kafka: Application events published to Kafka topics by Application Service. Consumer groups: Email Consumer Group, SMS Consumer Group, Push Notification Group. Each consumer group can scale independently. User preference: Check user notification preferences (opt-out, channel preference) before sending. Deduplication: Use Bloom filter or Redis to deduplicate events — at-least-once delivery means duplicates are possible. Email infrastructure: Use a provider like SendGrid/SES with retry logic. Observability: Track delivery rates, bounce rates, open rates per notification type.

Q3.

How would you handle the global scale — multiple data centers?

◆ **Model Answer**

Active-Passive: One primary region handles writes. Other regions replicate for reads. Active-Active: Both regions handle writes — complex conflict resolution needed. For Job Portal, I'd recommend Active-Passive for Phase 1. Use a global load balancer (AWS Route53 with latency routing) to direct users to the nearest region. Data replication: MySQL read replicas cross-region. Eventual consistency for read models (Elasticsearch, Redis) — acceptable for job searches. Fail-over: Automatic DNS failover to secondary region if primary is unhealthy. CDN: Static assets and cached API responses served from 50+ edge locations globally.

Q4.

How would you ensure the system achieves 99.9% uptime (SLA)?

◆ **Model Answer**

99.9% = 8.7 hours downtime/year max. 99.99% = 52 minutes. Strategies: Eliminate single points of failure: Multiple instances of every service behind a load balancer. Multi-AZ database deployment (primary + standby in different availability zones). Circuit breakers: Degraded mode instead of total failure (show cached results if search is down). Chaos engineering: Regularly test failure scenarios (kill a pod, cut DB connection). SLOs and error budgets: Define what 99.9% means per service, track error

rates. Runbooks: Document every known failure mode and resolution steps. On-call rotation with escalation policy. Post-mortems without blame after every incident.

■ SECTION 6: DEVOPS — CLOUD + SRE CONCEPTS

Q1.

What is the difference between blue-green and canary deployment?

◆ **Model Answer**

Blue-Green: Two identical environments (blue = current, green = new). Switch all traffic from blue to green at once. Instant rollback — just switch back. Downside: Double the infrastructure cost. Canary: Route a small percentage (1%, 5%) of traffic to the new version. Monitor. If healthy, gradually increase to 100%. Lower risk of full blast-radius for bugs. Kubernetes: Use Argo Rollouts or Flagger for automated canary with metric-based promotion. I implemented canary in K8s by having 9 pods running v1 and 1 pod running v2 — effectively 10% canary traffic.

Q2.

What are SLIs, SLOs, and SLAs? Give examples from a Job Portal context.

◆ **Model Answer**

SLI (Service Level Indicator): The metric you measure. Example: 'Percentage of job search requests returning in under 500ms'. SLO (Service Level Objective): Your internal target. Example: 'Job search SLI $\geq$ 99.5% over a rolling 30-day window'. SLA (Service Level Agreement): External contractual commitment with penalties. Example: 'We guarantee 99.9% API availability, or we provide service credits'. Error Budget: $100\% - \text{SLO}$. If $\text{SLO} = 99.5\%$, error budget = 0.5%. When budget is exhausted, freeze feature work and focus on reliability.

Q3.

What is Infrastructure as Code (IaC) and have you used any tools?

◆ **Model Answer**

IaC treats infrastructure (servers, networks, load balancers) as code — version-controlled, reviewable, repeatable. Tools: Terraform (cloud-agnostic, declarative), AWS CloudFormation (AWS-specific), Pulumi (imperative IaC in Python/TypeScript). In my project, I used Helm Charts as IaC for Kubernetes — templated YAML manifests for all services. Benefits: Reproducible environments (dev == staging == prod), peer-reviewable infra changes, disaster recovery — can recreate entire infrastructure from code in minutes.

Q4.

What is AWS API Gateway and how does it compare to Spring Cloud Gateway?

◆ **Model Answer**

AWS API Gateway: Managed service — no servers to manage. Handles auth (AWS Cognito, Lambda Authorizer), throttling, caching, monitoring out of the box. Best for serverless architectures. Spring Cloud Gateway: Self-managed, runs in your K8s cluster. More control, custom filters, deep Spring integration. Best for traditional microservices deployments. In my project, I used Spring Cloud Gateway because we had full control of the infrastructure and needed custom JWT validation and rate-limiting logic.

Q5.

How would you implement secrets management in a production environment?

◆ **Model Answer**

Never hard-code secrets in code or environment variables in plain text in K8s manifests. Options: (1) AWS Secrets Manager / HashiCorp Vault — store secrets centrally, rotate automatically, fine-grained access control. (2) K8s Secrets with etcd encryption at rest (minimal but better than ConfigMaps). (3) Sealed Secrets (Bitnami) — encrypt K8s Secrets for safe storage in Git. In Spring Boot: `spring.config.import=aws-secretsmanager:` or `vault://` to fetch secrets at startup. Implement secret rotation without downtime by restarting pods in a rolling update.

■ SECTION 7: CODING ROUND — MEDIUM TO HARD

Problem 1 — Merge Intervals (Medium)

Given intervals `[[1,3],[2,6],[8,10],[15,18]]`, merge overlapping ones → `[[1,6],[8,10],[15,18]]`.

```
public int[][] merge(int[][] intervals) { if (intervals.length <= 1) return intervals; Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0])); List<int[]> merged = new ArrayList<>(); int[] current = intervals[0]; for (int i = 1; i < intervals.length; i++) { if (intervals[i][0] <= current[1]) { // Overlapping: extend current interval current[1] = Math.max(current[1], intervals[i][1]); } else { merged.add(current); current = intervals[i]; } } merged.add(current); return merged.toArray(new int[0][]); }
```

$O(n \log n)$ for sort + $O(n)$ merge. Follow-up: Insert a new interval into sorted list?

Problem 2 — Top K Frequent Elements (Medium)

Given `[1,1,1,2,2,3]` and `k=2`, return `[1,2]` (the 2 most frequent elements).

```
public int[] topKFrequent(int[] nums, int k) { // Count frequencies Map<Integer, Integer> freqMap = new HashMap<>(); for (int n : nums) freqMap.merge(n, 1, Integer::sum); // Min-heap of size k: O(n log k) PriorityQueue<Integer> heap = new PriorityQueue<>((Comparator.comparingInt(freqMap::get))); for (int num : freqMap.keySet()) { heap.offer(num); if (heap.size() > k) heap.poll(); // remove least frequent } // O(k log k) to extract int[] result = new int[k]; for (int i = k - 1; i >= 0; i--) result[i] = heap.poll(); return result; }
```

$O(n \log k)$ — better than $O(n \log n)$ sort. Follow-up: Bucket sort approach for $O(n)$?

Problem 3 — Word Search in Grid (Medium-Hard)

Given an `m x n` grid of characters and a target word, return true if the word exists in the grid (adjacent cells horizontally/vertically).

```
public boolean exist(char[][] board, String word) { int m = board.length, n = board[0].length; for (int i = 0; i < m; i++) for (int j = 0; j < n; j++) if (dfs(board, word, i, j, 0)) return true; return false; } private boolean dfs(char[][] board, String word, int r, int c, int idx) { if (idx == word.length()) return true; if (r < 0 || r >= board.length || c < 0 || c >= board[0].length || board[r][c] != word.charAt(idx)) return false; char temp = board[r][c]; board[r][c] = '#'; // mark visited boolean found = dfs(board, word, r+1, c, idx+1) || dfs(board, word, r-1, c, idx+1) || dfs(board, word, r, c+1, idx+1) || dfs(board, word, r, c-1, idx+1); board[r][c] = temp; // restore return found; }
```

*DFS with backtracking. Modify board in-place to avoid visited set. $O(m*n*4^L)$ worst case.*

Problem 4 — Design Rate Limiter (Hard — Design Question)

Design a rate limiter that allows `N` requests per second per user. Implement the core logic.

```
// Sliding Window Log approach: public class RateLimiter { private final int maxRequests; private final long windowMillis; // userId -> timestamps of requests in window private final Map<String, Deque<Long>> userTimestamps = new
```

```
ConcurrentHashMap<>()); public RateLimiter(int maxRequests, long windowMs) {
this.maxRequests = maxRequests; this.windowMillis = windowMs; } public
synchronized boolean isAllowed(String userId) { long now =
System.currentTimeMillis(); Deque<Long> timestamps = userTimestamps
.computeIfAbsent(userId, k -> new ArrayDeque<>()); // Remove timestamps outside
the window while (!timestamps.isEmpty() && now - timestamps.peekFirst() >=
windowMillis) { timestamps.pollFirst(); } if (timestamps.size() < maxRequests) {
timestamps.addLast(now); return true; // allowed } return false; // rate limited }
}
```

For distributed rate limiting, use Redis with INCR + EXPIRE or the Token Bucket in Redis Lua scripts.

■ SECTION 8: RAPID FIRE — TRICKY CROSS QUESTIONS

Q1	Is @Transactional on a private method effective?	No! Spring AOP creates a proxy — private methods are not proxied. @Transactional only works on public methods called from outside the bean.
Q2	Can you inject a prototype bean into a singleton bean correctly?	Not with simple @Autowired — singleton gets one prototype instance at creation. Use ApplicationContext.getBean() or @Lookup method injection to get a new prototype each time.
Q3	What is spring.jpa.ddl-auto and what's the danger of 'update'?	'update' changes schema incrementally but doesn't drop removed columns — schema drift. Use 'validate' in production + Flyway for actual migrations.
Q4	Difference: ConcurrentHashMap vs Collections.synchronizedMap()? 	ConcurrentHashMap: Segment-level locking, high concurrency. synchronizedMap: Locks entire map on every operation — bottleneck under high concurrency.
Q5	What is method hiding vs method overriding?	Overriding: instance methods — runtime polymorphism. Hiding: static methods — compile-time binding. Calling via superclass reference calls superclass static method.
Q6	What does 'var x = new HashMap<>()' resolve to in Java 10?	HashMap — the compiler infers the specific type from the RHS. But type argument is inferred from context.
Q7	What is a phantom read in database transactions?	When a transaction reads a set of rows, another transaction inserts a row matching the criteria, and the first transaction reads again and sees the new row. Prevented by SERIALIZABLE isolation.
Q8	Can you have multiple @Primary beans?	No — having two @Primary beans for the same type causes NoUniqueBeanDefinitionException. Use @Qualifier to disambiguate instead.
Q9	What is HATEOAS?	Hypermedia As The Engine Of Application State. REST responses include links to related actions. Spring HATEOAS library. Example: job response includes {apply: '/jobs/1/apply'}.
Q10	What is idempotency of PUT vs PATCH?	PUT is idempotent — sending the same request N times has the same result. PATCH may or may not be idempotent depending on semantics (increment counter PATCH is not idempotent).
Q11	What happens if @Transactional method calls another @Transactional method in the SAME class?	Self-invocation bypasses the proxy — the second method's transaction annotation is IGNORED. Both run in the first method's transaction (or no transaction if no outer one exists).
Q12	What is a fat JAR in Spring Boot?	An executable JAR containing all dependencies (including an embedded Tomcat server). Created by spring-boot-maven-plugin. Run with: java -jar app.jar.
Q13	How does Kubernetes know when to restart a crashed container?	The container process exits with a non-zero code OR the liveness probe fails consecutively (failureThreshold times). K8s restarts per the RestartPolicy (default: Always).
Q14	What is SQL HAVING vs WHERE?	WHERE filters rows BEFORE aggregation. HAVING filters groups AFTER aggregation. WHERE cannot reference aggregate functions; HAVING can.

Q15 What is eventual consistency and when is it acceptable?

Data propagates to all nodes eventually, not immediately. Acceptable when reads can be slightly stale (search results, recommendations). NOT acceptable for financial transactions or seat bookings.

■ SECTION 9: EVALUATION GUIDE — SENIOR LEVEL EXPECTATIONS

What Separates a Good Candidate from an Exceptional One

- Systems Thinking: Doesn't just answer 'how' — answers 'why this design over alternatives' and 'what breaks at scale'.
- Production Mindset: Mentions monitoring, alerting, graceful degradation, and runbooks unprompted.
- Trade-off Fluency: Every answer includes 'but the tradeoff is...' — shows mature engineering judgment.
- Learning Agility: Cites recent reading, experiments, or certifications — actively grows.
- Communication: Can explain a complex distributed system clearly to both technical and non-technical audiences.
- Ownership: Says 'I decided...' 'I debugged...' 'I proposed...' — not 'we always did...'

Final Preparation Checklist for Viraj

- Practice explaining your Job Portal architecture in under 3 minutes — fluent and confident.
- Have 5 STAR stories ready: challenge, conflict, leadership, failure, improvement.
- Know your top 3 strengths with technical evidence from your project.
- Review Java 8 streams, Optional, Comparator — write code on paper.
- Practice SQL queries: GROUP BY, HAVING, window functions, subqueries.
- Know Spring @Transactional pitfalls (self-invocation, private methods, propagation).
- Prepare 3 intelligent questions to ask the interviewer.
- Rehearse out loud — not just reading. The difference is enormous.

Viraj, you have real project experience, a strong tech stack, and 3.5 years of growth at Capgemini. Own your story, trust your preparation, and walk into that Accenture interview with confidence. These 3 mock interviews cover everything you need. Best of luck! ■